[게임 제목 삽입] — 프로젝트 개요

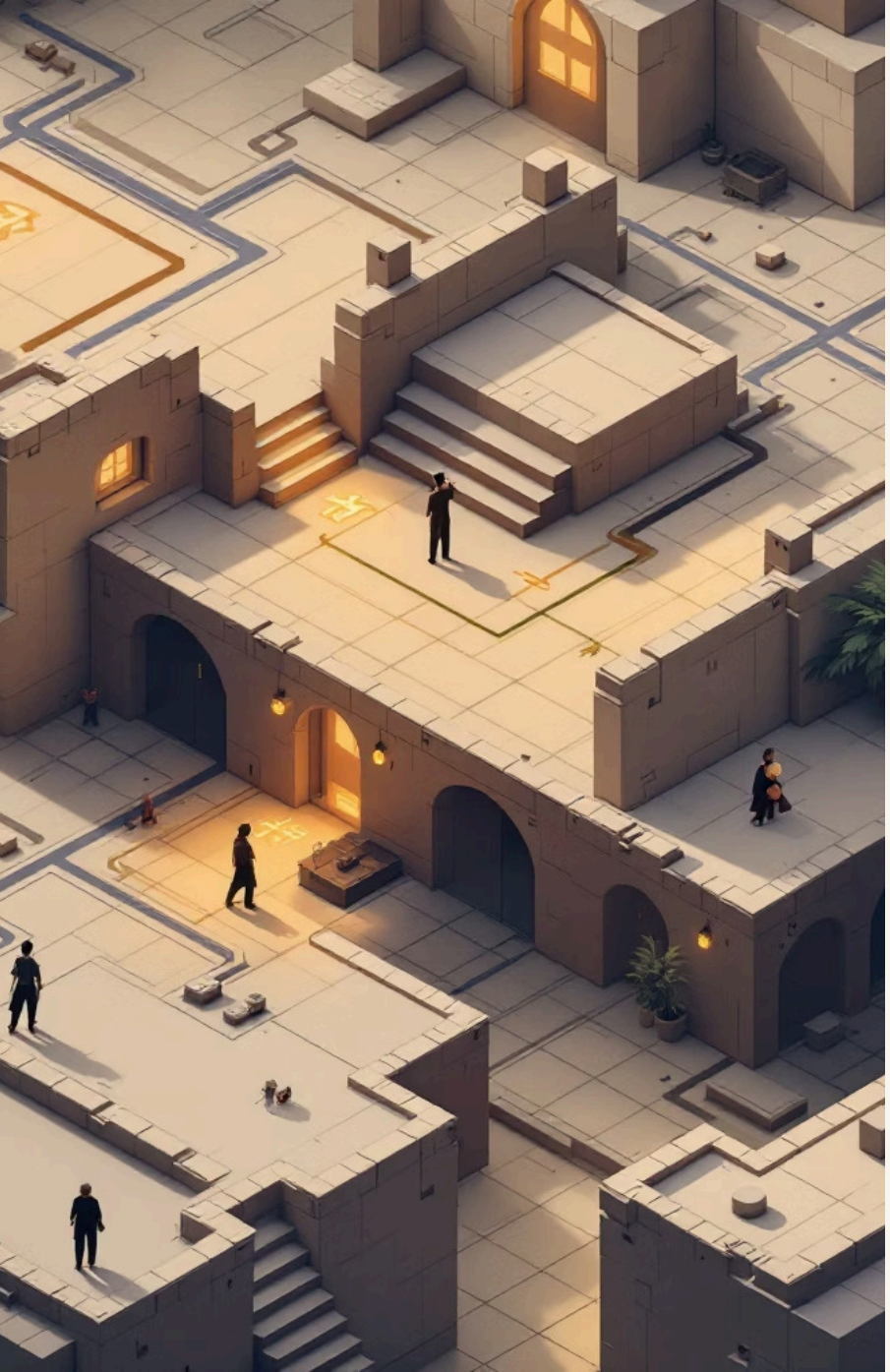
핵심 컨셉

JRPG의 전투 재미와 SRPG의 전략적 이동을 결합한 **턴제 게임**

개발 목표

- 마을-필드-던전으로 이어지는 유기적인 게임 루프 구축
- 확장성 있는 컴포넌트 기반 아키텍처 및 데이터 구동 시스템 구현

Made with GAMMA



핵심 기술 1 — 이동 및 길찾기 시스템

> "그리드 기반의 전략적 경로 탐색 구현"

TileMap A* Pathfinding

- `TileMap::FindPath` 로직을 통한 타일 인덱스 기반 최적 경로 계산
- 벽, 몬스터 등 장애물과의 충돌 체크(`CanMove`, `CheckPortal`) 실시간 처리

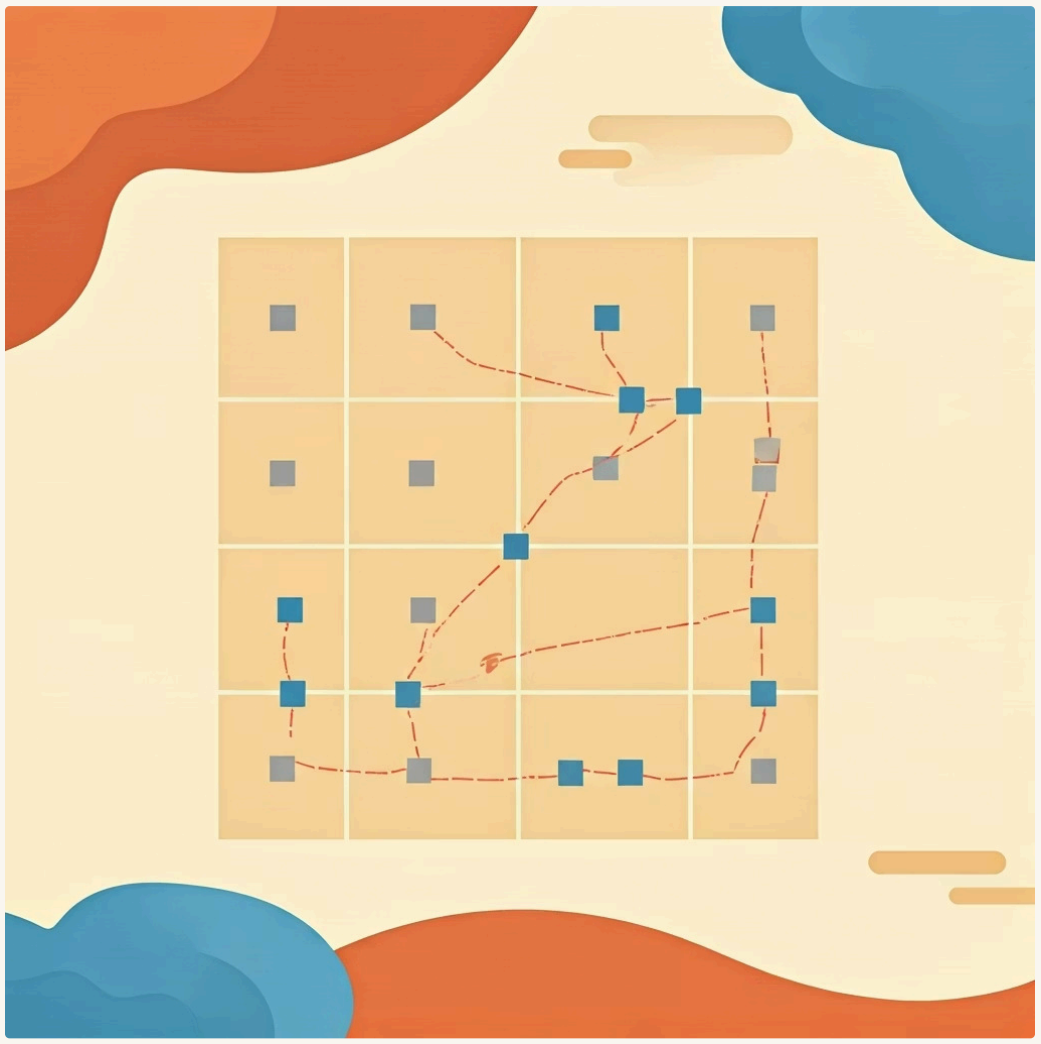
이동 속성 및 시각화

- Actor별 `Range`(이동), `Sight`(시야) 데이터를 기반으로 이동 가능 영역 하이라이트
- 커서 위치에 따른 실시간 경로 가이드 및 `isMoving`, `fMoveSpd`를 이용한 부드러운 이동 연출

Made with GAMMA

A* 경로 탐색 알고리즘 구현

최적 경로를 효율적으로 찾는 휴리스틱 기반 알고리즘



핵심 개념

- **g(n):** 시작점에서 현재 노드까지의 실제 거리
- **h(n):** 현재 노드에서 목표점까지의 예상 거리 (휴리스틱)
- **f(n) = g(n) + h(n):** 총 비용 (가장 낮은 f값을 우선 탐색)

구현 특징

- 벽과 장애물 충돌 체크 (CanMove, CheckPortal)
- 4방향 또는 8방향 이동 지원
- 우선순위 큐를 통한 효율적인 노드 관리

구현 세부사항

```
    Vector2 pos;
    float g, h;
    Node* parent;
    float f() const { return g + h; }
};

std::vector<Vector2> path;
if (!IsWalkable((int)end.x, (int)end.y)) return path;

std::vector<Node*> openList;
std::vector<Node*> closedList;

Vector2 startTemp;
Node* startNode = new Node{ start, 0, startTemp.Distance(start, end), nullptr };
openList.emplace_back(startNode);

while (!openList.empty())
{
    // F값이 가장 낮은 노드 선택
    auto it = std::min_element(openList.begin(), openList.end(),
        [](Node* a, Node* b) { return a->f() < b->f(); });

    Node* current = *it;
    if (current->pos == end)
    {
        // 경로 복원
        while (current)
        {
            path.emplace_back(current->pos);
            current = current->parent;
```

```
    Vector2 neighbors[] = { {0,-1}, {0,1}, {-1,0}, {1,0} };
    for (auto& dir : neighbors)
    {
        Vector2 nextPos = current->pos + dir;
        if (!IsWalkable((int)nextPos.x, (int)nextPos.y)) continue;

        auto isClosed = std::find_if(closedList.begin(), closedList.end(),
            [&](Node* n) { return n->pos == nextPos; });
        if (isClosed != closedList.end()) continue;

        float newG = current->g + 1;
        auto isOpen = std::find_if(openList.begin(), openList.end(),
            [&](Node* n) { return n->pos == nextPos; });

        Vector2 temp;
        if (isOpen == openList.end())
        {
            openList.emplace_back(new Node{ nextPos, newG,  temp.Distance(nextPos, end), current });
        }
        else if (newG < (*isOpen)->g)
        {
            (*isOpen)->g = newG;
            (*isOpen)->parent = current;
        }
    }
}

// 메모리 정리
for (auto n : openList) delete n;
```

```
if (m_eFieldPhase == FieldState::PlayerTurn && m_bIsRangeCalculated)
{
    for (const auto& tilePos : m_vecMoveRangeTiles)
    {
        Vector2 screenPos = tilePos - renderSys.GetCamera().GetPosition();
        renderSys.GetWorldCanvas().DrawTxt(screenPos.x, screenPos.y, L"Wu2592", Color::BrightCyan,1);
    }
}

if (m_eFieldPhase == FieldState::EnemyTurn ||
    m_eFieldPhase == FieldState::EnemyMove ||
    m_vecMonsterRangeTiles.empty() == false)
{
    for (const auto& vPos : m_vecMonsterRangeTiles)
    {
        Vector2 vScreen = renderSys.GetCamera().WorldToScreen(vPos);
        // 붉은색 쉐이드 타일로 위험 지역 표시
        renderSys.GetWorldCanvas().DrawTxt((int)vScreen.x, (int)vScreen.y, L"Wu2591", Color::Red,1);
    }
```

```
int iRange = pTarget->GetMoveRange();
Vector2 vStart = pTarget->GetPosition();

m_vecMonsterRangeTiles.clear();

Vector2 startPos = pTarget->GetPosition();
int maxRange = pTarget->GetMoveRange();

// BFS를 위한 큐와 방문 체크 맵
std::queue<std::pair<Vector2, int>> q;
```

```
    m_vecMonsterRangeTiles.push_back(vCurr);

if (iDist >= iRange)
    continue;

Vector2 vDirs[] = { {0,1}, {0,-1}, {1,0}, {-1,0} };
for (const auto& vDir : vDirs)
{
    Vector2 vNext = vCurr + vDir;
    // 맵 안이고, 갈 수 있는 타일인 경우
    if (m_worldMap->IsWalkable((int)vNext.x, (int)vNext.y))
    {
        if (visited.find({ (int)vNext.x, (int)vNext.y }) == visited.end())
```

Made with GAMMA

핵심 기술 2 — 전투 아키텍처

> "책임 분리를 통한 확장성 있는 전투 엔진 설계"

BattleContext (데이터 및 통로)

- 전투의 현재 상태(BattleState)와 참여 액터 정보를 관리하는 데이터 허브
- 이벤트(데미지, 연출)가 전달되는 중심 통로 역할

BattleResolver (연산 엔진)

- 데미지 계산, 상태 이상(StatusEffect) 판정 등 순수 로직 전담
- Context와 분리되어 독립적인 단위 테스트 및 로직 수정 용이

BattleState FSM

TurnCheck → Command → Select → Animation → Result로 이어지는 엄격한 상태 관리를 통해 턴제 로직 안정성 확보

Made with GAMMA

아키텍처 및 구현 다이어그램

BattleEventFactory 클래스 다이어그램 - 이벤트 생성의 책임을 분리하는 팩토리 패턴 구조

```
class Actor;
class Vector2;
class IBattleEventFactory
{
public:
        virtual ~IBattleEventFactory() = default;
        virtual std::unique_ptr<ICutsceneEvent> CreateAsciiAnimation(Actor* pOwner, EAniType eType, float fFrameDelay = 0.4f)
        virtual std::unique_ptr<ICutsceneEvent> CreateAsciiParticle(const Vector2& pos, int iParticleCnt, float fDuration) =
        virtual std::unique_ptr<ICutsceneEvent> CreatePhaseChange(BattleState nextState) = 0;
        virtual std::unique_ptr<ICutsceneEvent> CreateDmg(Actor* pAtker, Actor* pTarget,
            int iDmg, bool bCritical) = 0;
        virtual std::unique_ptr<ICutsceneEvent> CreateDeath(Actor* pTarget) = 0;
        virtual std::unique_ptr<ICutsceneEvent> CreateDelay(float fDeltaTime) = 0;
        virtual std::unique_ptr<ICutsceneEvent> CreateLog(const BattleLog& log) = 0;
        virtual std::unique_ptr<ICutsceneEvent> CreateMove(Actor* pActor, const Vector2& targetPos, float fDuration) = 0;
        virtual std::unique_ptr<ICutsceneEvent> CreateVisualEffect(Vector2 start, Vector2 end,
            EffectMovementType eType, wchar_t txt, Color color, float fDuration) = 0;
        virtual std::unique_ptr<ICutsceneEvent> CreateTurnEnd(Actor* actor) = 0;
        virtual std::unique_ptr<ICutsceneEvent> CreateTurnStart(Actor* actor) = 0;
};
```

팩토리 패턴의 상세 구현 흐름 - MoveEvent와 AsciiAniEvent 생성 프로세스

```
std::unique_ptr<Wannabe::ICutsceneEvent> GameBattleEventFactory::CreateAsciiAnimation(Wannabe::Actor* pOwner,
{
    return std::make_unique<AsciiAnimationEvent>(pOwner, eType, fFrameDelay);
}

std::unique_ptr<Wannabe::ICutsceneEvent> GameBattleEventFactory::CreateAsciiParticle(const Wannabe::Vector2& p
{
    return std::make_unique<AsciiParticleEvent>(pos, iParticleCnt, fDuration);
}

std::unique_ptr<Wannabe::ICutsceneEvent> GameBattleEventFactory::CreatePhaseChange(BattleState nextState)
{
    return std::make_unique<BattlePhaseChangeEvent>(nextState);
}

std::unique_ptr<Wannabe::ICutsceneEvent> GameBattleEventFactory::CreateDmg(Wannabe::Actor* pAtker, Wannabe::Ac
{
    return std::make_unique<DamageEvent>(pAtker, pTarget, iDmg, bCritical);
}

std::unique_ptr<Wannabe::ICutsceneEvent> GameBattleEventFactory::CreateDeath(Wannabe::Actor* target)
{
    return std::make_unique<DeathEvent>(target);
}
```

Command → BattleContext의 상호작용 다이어그램

```
void SkillCommand::Execute(Wannabe::BattleContext& context)
{
    if (context.IsValidActor(m_pTarget) == false || context.IsValidActor(m_pInstigator) == false)
        return;

    auto& resolver = context.GetResolver();
    auto& processor = context.GetEventProcessor();

    const ActionData* skill = DataManager::Get().GetActionData(m_iSkillTid);
    if (skill == nullptr)
        return;

    ActCheckResult act = resolver.CanAct(m_pInstigator);
    if (act.bCanAct == false)
    {
        BattleLog log;
        log.eLogType = LogType::Free;
        log.wstrTxt = act.wstrReason;
        context.GetCutscenePlayer().Push(std::make_unique<LogEvent>(log));
        return;
    }

    int targetCnt = skill->iMaxTargetCnt;
    ActionTargetType targetType = skill->targetType;
    // 타겟 선정
    auto targets = context.GetResolver().ResolveTargets(context, m_pInstigator, m_pTarget, targetType, targe
    for (auto* target : targets)
    {
        auto results = resolver.ResolveAction(m_pInstigator, target, skill->vecEffects,skill->iPower);

        for (const auto& result : results)
            processor.ProcessCombatEffectResult(context, result);
    }
}
```

BattleResolver의 역할, 계산기

```
std::vector<CombatEffectResult> BattleResolver::ResolveAction(Actor* pAtker, Actor* pTarget, const std::vector<CombatEffectData>& da
{
    std::vector<CombatEffectResult> results;

    if (IsValidActor(pAtker) == false || IsValidActor(pTarget) == false)
        return results;

    CombatEffectResult result;
    for (const auto& effectData : data)
    {
        CombatEffect effect;
        effect.pAtker = pAtker;
        effect.pTarget = pTarget;
        effect.eCombatEffectType = effectData.eType;
        if (CheckMiss(pAtker, pTarget)) { ... }

        switch (effectData.eType)
        {
        case CombatEffectType::Damage:
        {
            int damage = CalcSkillDamage(pAtker,pTarget,iPower,effectData.iRatio);
            if (CheckCritical(pAtker, pTarget))// 크리티컬 판정
            {
                damage = static_cast<int>(damage * 1.2f);
                effect.bCritical = true;
            }

            effect.iValue = damage;
            break;
        }
```

BattleEventProcessor의 역할, 실행기(TurnCheck → Command → Context→Resolver→ "Processor")

```
void BattleEventProcessor::ApplyDmg(BattleContext& context, const CombatEffect& effect)
{
    Actor* atker = effect.pAtker;
    Actor* target = effect.pTarget;
    if (atker == nullptr || target == nullptr)
        return;

    auto stat = target->GetComponent<StatComponent>();
    if (stat == nullptr)
        return;

    if(effect.bMiss == true) { ... }

    int dmg = stat->ApplyDmg(effect.iValue);

    Vector2 startPos = atker->GetPosition();
    Vector2 endPos = target->GetPosition();

    // 이동 연출
    context.GetCutscenePlayer().Push(m_pEventFactory->CreateMove(atker, endPos, 0.3f));

    // 공격자 애니 (atk타입)
    context.GetCutscenePlayer().Push(m_pEventFactory->CreateAsciiAnimation(atker, EAniType::Attack));

    // 투사체 연출, 직선
    context.GetCutscenePlayer().Push(m_pEventFactory->CreateVisualEffect(
        startPos, endPos, EffectMovementType::Parabola, L'*', Color::Yellow, 0.8f
    ));

    // 피격자 애니메이션
    context.GetCutscenePlayer().Push(m_pEventFactory->CreateAsciiAnimation(target, EAniType::Hit));

    // 로그 출력
    BattleLog log;
    log.wstrAtkerName = atker->GetComponent<DisplayComponent>()->GetOriginName();
    log.wstrTargetName = target->GetComponent<DisplayComponent>()->GetOriginName();
    log.eLogType = LogType::Damage;
    log.iValue = dmg;
    log.bCritical = effect.bCritical;
    log.bMiss = effect.bMiss;
    context.GetCutscenePlayer().Push(m_pEventFactory->CreateLog(log));
```

Made with GAMMA

핵심 기술 3 — 이벤트 기반 연출

> "ASCII 환경을 극복하는 동적 연출 시스템"

Factory Pattern 기반 이벤트 생성

IBattleEventFactory를 통해 MoveEvent(왕복 이동), AsciiAniEvent(스프라이트 재생) 등을 동적 관리

몰입감 강화 요소

→ **Camera Shake**

데미지 발생 시 RenderSystem 트리거를 통한 화면 흔들림

→ **ANSI Color**

보라색 등 다양한 ANSI 코드를 활용한 시각적 효과 극대화

→ **Matrix 효과**

전투 진입 시 시그니처 연출 구현

Made with GAMMA

AsciiAnimationEvent의 h/cpp 구현 내용

```
class AsciiAnimationEvent : public Wannabe::ICutsceneEvent
{
public:
    AsciiAnimationEvent(Wannabe::Actor* pOwner, EAniType eType, float fFrameDelay);

    virtual void OnStart(Wannabe::BattleContext& context) override;
    virtual bool Update(Wannabe::BattleContext& context, float fDeltaTime) override;
    virtual bool IsFinished() const override { return m_bIsFinished; }
    virtual CutsceneEventType GetEventType() override
    {
        m_eCutsceneEventType = CutsceneEventType::AsciiAnimation;
        return m_eCutsceneEventType;
    };

private:
    void DrawBox(Wannabe::RenderSystem& renderSys);

private:
    Wannabe::Actor* m_pOwner = nullptr;
    EAniType m_eType;
    std::vector<std::vector<std::wstring>> m_frames;
    float m_fFrameDelay = 0.1f;
    float m_fElapsed = 0.f;
    int m_iCurrentFrame = 0;
    bool m_bIsFinished = false;

    // 중앙 박스 규격 (포켓몬 스타일)
    const int BOX_WIDTH = 30;
    const int BOX_HEIGHT = 10;
    const Wannabe::Vector2 BOX_POS = { 55, 12 }; // 140x40 기준 중앙 부근
};
```

```
bool AsciiAnimationEvent::Update(Wannabe::BattleContext& context, float fDeltaTime)
{
    if (m_bIsFinished)
        return false;

    m_fElapsed += fDeltaTime;
    if (m_fElapsed >= m_fFrameDelay) { ... }

    // 중앙 상자
    auto& renderSys = Wannabe::Engine::Get().GetRenderSystem();
    DrawBox(renderSys);

    // 현재 프레임 아스키 출력
    const auto& currentArt = m_frames[m_iCurrentFrame];
    int startY = BOX_POS.y + (BOX_HEIGHT - (int)currentArt.size()) / 2; //세로 중앙 정렬
    for (int i = 0; i < currentArt.size(); ++i)
    {
        int textWidth = (int)currentArt[i].length();
        int offsetX = (BOX_WIDTH - textWidth) / 2;

        Wannabe::Vector2 drawPos;
        drawPos.x = BOX_POS.x + offsetX;
        drawPos.y = startY + i;

        renderSys.DrawUI(currentArt[i], drawPos, Wannabe::Color::BrightCyan, 200);
    }

    return true;
}

void AsciiAnimationEvent::DrawBox(Wannabe::RenderSystem& renderSys)
{
    // 테두리 및 배경 소거 (포켓몬 대화창 느낌)
    std::wstring horizontal = L"+" + std::wstring(BOX_WIDTH - 2, L'-') + L"+";
    std::wstring emptyLine = L"|" + std::wstring(BOX_WIDTH - 2, L' ') + L"|";

    renderSys.DrawUI(horizontal, BOX_POS, Wannabe::Color::BrightGold, 190);
    for (int i = 1; i < BOX_HEIGHT - 1; ++i)
        renderSys.DrawUI(emptyLine, { BOX_POS.x, BOX_POS.y + i }, Wannabe::Color::BrightGray, 190);
    renderSys.DrawUI(horizontal, { BOX_POS.x, BOX_POS.y + BOX_HEIGHT - 1 }, Wannabe::Color::BrightGold, 190);
}
```

Made with GAMMA

프로젝트 타임라인 (3/4 ~ 3/10)

날짜	단계	실행 내역 요약
3/4	**기획**	JRPG → SRPG 확장 설계 및 마을-필드-던전 순환 구조 확립
3/5~6	**기반 구축**	Level 전환 지연 생성 로직, JSON 기반 Save/Load 및 TileMap 시스템 추가
3/7	**안정화**	엔진 강제 종료 시 메모리 누수 방지 로직 및 휘발성 BattleLevel 도입
3/8	**데이터/UI**	인벤토리/아이템 시스템 연동, MenuList와 TargetCursor 통합 관리
3/9	**AI/길찾기**	A* 경로 탐색 적용, 필드 이동 로직 분기 및 BattleState 세분화
3/10	**연출/정리**	MoveEvent, AsciiAniEvent 추가, Resolver 책임 재정리 및 리팩토링

Made with GAMMA

트러블슈팅 (Troubleshooting)

🐛 이슈: 틱(Tick) 중 레벨 전환 시 댕글링 포인터 발생

원인: 프레임 갱신 중에 레벨이 바뀌면서 참조 중인 객체가 소멸됨.

해결: `OnFrameEnd` 단계를 추가하여 모든 연산 종료 후 안전하게 레벨을 교체하는 **'지연 생성'** 방식 도입.

🐛 이슈: 엔진 강제 종료 시 메모리 누수

원인: 소멸자가 정상 호출되지 않는 환경에서 리소스 잔류.

해결: 종료 시퀀스(애니메이션)를 강제하여 소멸 로직이 반드시 실행되도록 보장.

Made with GAMMA

회고 및 자체 평가

완성도 점수

9 / 10

성과

- 객체 지향적 설계(컴포넌트 추상화)를 통해 유지보수성이 높은 코드 베이스 구축
- 짧은 기간 내에 단순 전투에서 전략적 이동이 포함된 SRPG로의 전환 성공

아쉬운 점 및 보완

- 아이템, 스킬 관련 가시화
- 마을 기능 구현

Made with GAMMA